



Text To Image Generation StackGAN

Computational Intelligence

Author: Maša Cucić

September 26, 2024.

Contents

1	Introduction	2
2	Dataset	2
3	Methodology	3
3.1	Conditioning Augmentation Network	3
3.2	Stack GAN	4
3.2.1	Stage 1	4
3.2.2	Stage 2	5
3.3	Loss Functions	5
3.3.1	Generator Loss	5
3.3.2	Discriminator Loss	6
4	Training Phase	6
4.0.1	Training on a Single Bird Species	6
4.0.2	Training on 40 Bird Species	7
5	Testing Phase	9
6	Results and Conclusion	9

1 Introduction

The topic of this paper is image generation based on textual descriptions, with a focus on generating images of birds.

Several approaches exist for solving this problem using GANs. A Generative Adversarial Network (GAN) is a deep learning model consisting of two adversarial models: a generator and a discriminator. The generator and discriminator are trained together. The goal of the generator is to learn how to create realistic data, while the discriminator attempts to distinguish fake data (generated by the generator) from real data. In this way, the two models mutually improve each other.

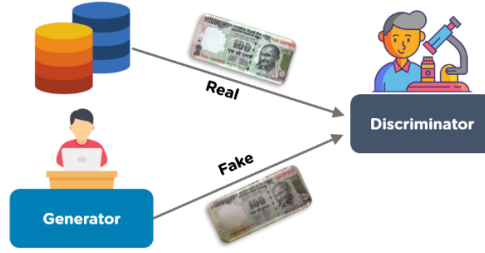


Figure 1: GAN

This work is based on the StackGAN approach. This is a more advanced method than traditional GANs, featuring a hierarchical architecture. Training is conducted in two phases, each consisting of a generator and a discriminator. In the first phase, low-resolution images are generated, which are then refined to higher resolution in the subsequent phase. Implementation details of StackGAN will be discussed later in the text.

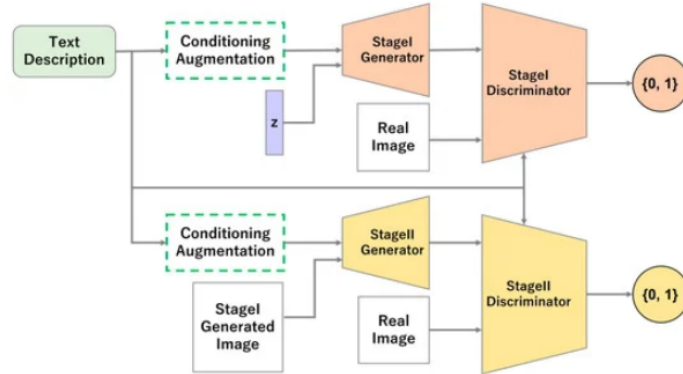


Figure 2: StackGAN

2 Dataset

The model was trained using the CUB-200-2011 dataset (Caltech-UCSD Birds-200-2011). The dataset was downloaded from the link provided in the references [1].

This dataset contains 200 different bird species. For each species, there are approximately 60 images of that bird type, along with 10 textual descriptions for each

image, with a total of over 11,000 images. Due to technical limitations, the experimental work utilized several subsets of this data. The primary focus was on training 40 bird species, as well as training on a single species.

The split into training and test sets was already established by downloading the corresponding textual embeddings. Within the train and test folders, there are three files each:

- `char-CNN-RNN_embeddings.pickle` - contains vector representations of the image descriptions
- `class_info.pickle` - contains class information for each image
- `filenames.pickle` - contains a list of all image filenames

By downloading these files, the split into training and test sets was automatically completed. The training and test sets is 75:25.

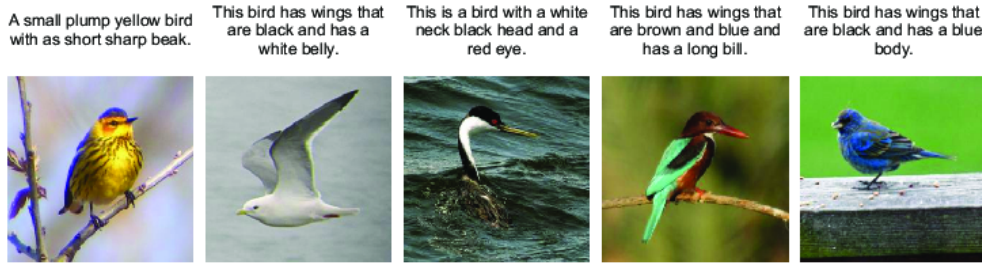


Figure 3: CUB_200_2011

3 Methodology

3.1 Conditioning Augmentation Network

The Conditioning Augmentation Network (CANet) plays a key role in the StackGAN model for generating high-quality bird images from textual descriptions. It transforms textual embeddings into conditional vectors, that the generator and discriminator can use more effectively to synthesize images based on those descriptions.

Conditioning Augmentation is a technique used to increase the diversity of the conditional variables employed for data generation. Instead of relying on fixed conditions (e.g. raw textual embeddings), this technique randomly samples latent variables ($\mathbf{c}$) from an independent normal distribution. The goal is to enhance the diversity during training and improve the model’s robustness to small variations in the conditioning input.

First, via an encoding function, the text embedding is projected into a higher-dimensional space using a linear transformation. The result of this projection is split into two parts: $\boldsymbol{\mu}$ (mean) and $\log \boldsymbol{\sigma}^2$ (log-variance). The mean represents the center of the distribution of possible vectors, while the log-variance determines the width of this distribution, i.e., how far values can vary around the mean.

Then, using the reparameterization trick, a conditional vector $\mathbf{c}$ is generated, representing a random variation created by combining the mean with noise scaled by the standard deviation. Finally, the network returns three values: the conditional vector $\mathbf{c}$, the mean $\boldsymbol{\mu}$, and the log-variance $\log \boldsymbol{\sigma}^2$. This enables the model to generate more diverse images from the same textual descriptions.

3.2 Stack GAN

StackGAN (Stacked Generative Adversarial Networks) is an advanced approach in generative adversarial networks that employs a hierarchical, two-stage architecture to generate high-quality images from textual descriptions. The core process unfolds in two distinct stages, each dedicated to a specific phase of image generation and quality assessment. Both stages consist of a dedicated generator and discriminator pair.

In the first stage, the generator produces low-resolution images (64x64) that capture the fundamental characteristics outlined in the textual description. Simultaneously, the discriminator evaluates these images, learning to distinguish between real and generated samples.

In the second stage, the generator refines the images obtained from the first stage to produce higher-resolution outputs (256x256), incorporating finer details and greater visual fidelity. The discriminator of the second stage then assesses the quality of these enhanced images, operating similarly to its counterpart in Stage I.

During training, the generator and discriminator engage in an adversarial competition: the generator strives to create images that are increasingly realistic, while the discriminator aims to improve its ability to differentiate real images from generated ones. This continuous adversarial process drives both models to progressively improve, ultimately resulting in the generation of high-quality, detailed images.

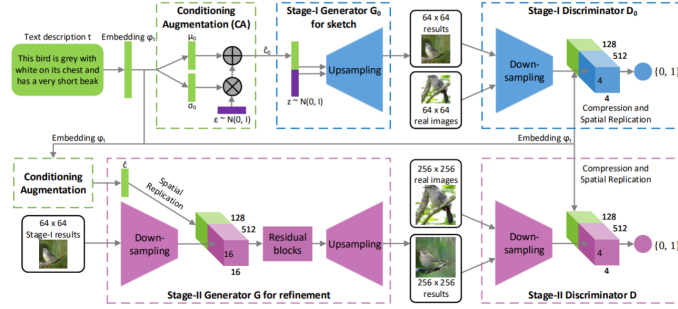


Figure 4: Stack GAN

3.2.1 Stage 1

In Stage 1, the main objective is to generate low-resolution images from textual descriptions that depict the basic characteristics of birds. The generator uses both textual embeddings and random noise to create images that represent the fundamental shapes and colors of birds, but with minimal detail. The inputs to the generator are:

- **text_embedding** — the corresponding textual embedding based on which the generator creates the images
- **noise** — random noise defined by `z_dim`, which represents the dimension of the latent space from which the noise is sampled

First, the textual description is transformed into a random conditional vector via the Conditioning Augmentation network. This vector is then combined with the provided noise. The generator passes through several layers, including an `UpSamplingBlock`, which uses a nearest-neighbor-based upsampling method to increase the spatial dimensions of the generated image, followed by `conv3x3` convolutional layers to refine image quality and detail.

The generator outputs both the generated image and the mean (μ) and log-variance ($\log \sigma^2$) of the latent space obtained from the CA network, which are needed for later stages. On the other hand, the discriminator takes the following inputs:

- **condition** — a conditioning vector
- **image** — the image to which this condition corresponds

For a given image, the discriminator must assess whether it is real or fake. In this stage, the discriminator evaluates the quality of the images using a deep convolutional architecture to determine if an image is real or fake. It encodes the input image, processes the condition, and fuses the image and condition into a single representation, based on which the discriminator outputs a probability score for the given image-condition pair. The discriminator returns the probability that the input image is real. A value closer to zero indicates that the discriminator considers the image fake, while a value closer to one indicates it believes the image is real.

3.2.2 Stage 2

The primary objective of Stage 2 is to generate higher-resolution images based on the results obtained from Stage 1. The generator and discriminator in this phase operate similarly to those in Stage 1, with one key difference in how the generator functions. In Stage 2, the generator utilizes information from the previous phase. This process works as follows: when the Stage 2 generator is instantiated, it must be provided with an instance of the Stage 1 generator. During image generation, the Stage 2 generator first uses the provided Stage 1 generator to produce a corresponding low-resolution image. It then encodes this image and combines the encoded features with other conditioning information to generate the new, higher-resolution image in Stage 2. The Stage 2 discriminator evaluates how the generated high-resolution images correspond to real images and provides feedback on their quality, operating similarly to the discriminator in Stage 1.

3.3 Loss Functions

3.3.1 Generator Loss

The generator loss measures how successfully the generator can deceive the discriminator. The generator’s objective is to produce images that appear realistic enough so that the discriminator cannot easily distinguish them from real images. The more convincing the generated images are, the harder it becomes for the discriminator to identify them as fake, leading to a lower generator loss.

When calculating losses, the Binary Cross-Entropy Loss (BCE) criterion is used. BCE Loss is employed to measure the difference between the actual and predicted classes in binary classification tasks. It is therefore suitable because it is applied to the discriminator’s outputs, which represent probabilities between 0 and 1, by comparing them with the corresponding class labels.

The generator loss is computed using the discriminator loss calculated for the fake images, comparing them with labels for real images. The discriminator’s output consists of values between 0 and 1 for each image in the batch, representing the probability that the image is real. A value closer to 1 indicates that the discriminator considers the image to be real. If the generator produces high-quality images, the discriminator will output probabilities close to 1 for those images, meaning it believes they are real. In this way, the computed loss for the discriminator becomes smaller, which contributes to a lower generator loss.

In addition to this loss, the total generator loss also includes KL divergence as a regularizer, which helps keep the latent space distribution stable, thereby improving the quality of the generated images.

3.3.2 Discriminator Loss

As previously explained, the BCE Loss criterion is used to compute the loss. The discriminator loss is calculated based on the losses for three types of pairs: real pairs, mismatched pairs, and fake pairs. Real pairs consist of real images paired with their corresponding correct conditions; mismatched pairs involve real images paired with randomly selected incorrect conditions; while fake pairs contain fake (generated) images paired with their corresponding conditions. The total discriminator loss is a weighted combination of these three loss values. The losses for mismatched and fake pairs are weighted to obtain a balanced evaluation, while the losses for real pairs dominate the total loss to ensure that the discriminator reliably recognizes real images and maintains training stability.

```
def generator_loss(netD, fake_imgs, real_labels, conditions, logvar):
    criterion = nn.BCELoss()
    cond = conditions.detach()
    fake = fake_imgs

    fake_features = netD(fake, cond)
    errD_fake = criterion(fake_features.view(-1), real_labels.view(-1))

    kl_loss = KL_loss(conditions, logvar)

    errG_total = errD_fake + kl_loss*2
    return errG_total
```

Figure 5: Generator Loss

```
def discriminator_loss(netD, real_imgs, fake_imgs, real_labels, fake_labels, conditions):
    criterion = nn.BCELoss()
    batch_size = real_imgs.size(0)

    cond = conditions.detach()
    fake = fake_imgs.detach()

    # Real parovi
    real_features = netD(real_imgs, cond)
    errD_real = criterion(real_features.view(-1), real_labels.view(-1))

    # Pogrešni parovi (realne slike uparene sa pogrešnim uslovima)
    wrong_cond = cond[torch.randperm(batch_size)] # slučajna permutacija
    wrong_features = netD(real_imgs, wrong_cond)
    errD_wrong = criterion(wrong_features.view(-1), fake_labels.view(-1))

    # Lažni parovi
    fake_features = netD(fake, cond)
    errD_fake = criterion(fake_features.view(-1), fake_labels.view(-1))

    errD = errD_real + (errD_fake + errD_wrong)*0.5
    return errD
```

Figure 6: Discriminator Loss

4 Training Phase

Each training epoch processes data in batches. Within each batch, the generator produces images based on the given data, aiming to make them as similar as possible to the real images from the dataset. The discriminator then evaluates these generated images by comparing them to the real images in the batch and attempts to distinguish between them. Based on this comparison, the losses for both models are computed: the **generator loss**, which indicates how close the generated images are to the real ones, and the **discriminator loss**, which reflects its ability to differentiate real images from fake ones.

At the end of each batch iteration, the weights of both the generator and discriminator are updated using the Adam optimizer with parameters $\text{betas} = (0.5, 0.999)$. The parameter $\text{beta}_1 = 0.5$ enables a faster response to gradient changes, while $\text{beta}_2 = 0.999$ retains its default value.

4.0.1 Training on a Single Bird Species

During training, I conducted an experiment to fine-tune the model for a single bird species. I concluded that better results are obtained when increasing the learning rate. Specifically, the initial learning rate was 0.0002 for both the generator and the discriminator. However, when training Stage 1 on a single species, the model failed to learn useful characteristics. By increasing the generator's learning rate to 0.0005 (and higher), the results achieved can be seen on Figures 7-11.



Figure 7: Real images

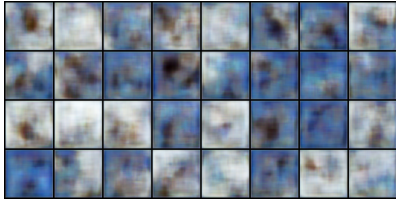


Figure 8: Stage 1

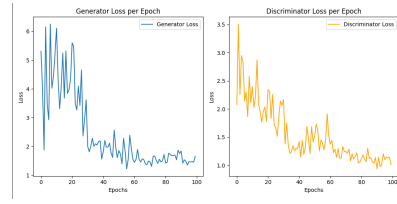


Figure 9: Stage 1 Loss

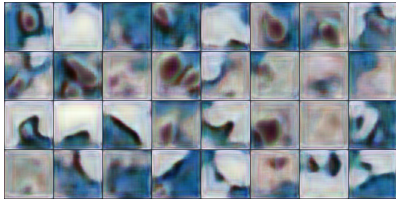


Figure 10: Stage 2

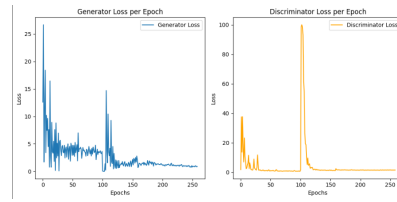


Figure 11: Stage 2 Loss

I concluded that the images in the second stage are somewhat more precise than those in the first stage, and that the colors are largely learned and accurately rendered. Furthermore, I successfully adjusted the parameters so that both the generator loss and the discriminator loss decrease, at least for the Stage 1.

4.0.2 Training on 40 Bird Species

The training on 40 bird species followed a procedure similar to that used for a smaller number of species. For a very low generator learning rate (below 0.0001), more abstract results were obtained for both 20 and 40 species. Unfortunately, no loss data is available for these results.

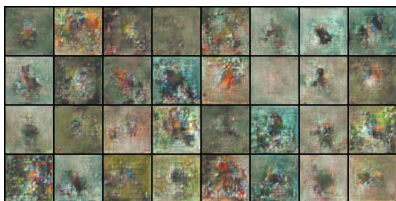


Figure 12: Stage 1, 20 species



Figure 13: Stage 1, 40 species

After this initial training, different results are obtained when using a higher learning rate. We can observe the outcomes of two consecutive training runs, where only the generator's learning rate was increased from 0.0002 to 0.0005.



Figure 14: Real Images, 40 species

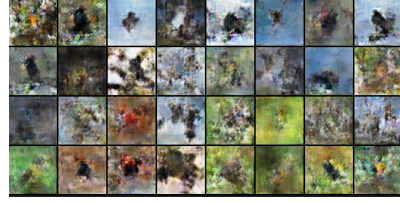


Figure 15: Stage 1, lr=0.0005



Figure 16: Real images, 40 species



Figure 17: Stage 1, lr=0.0002

I concluded that with a higher learning rate, the generator produces somewhat more precise images with clearer shapes. Unfortunately, in both cases, a situation arises where the discriminator overpowers the generator, which can be observed in the Figure 18.

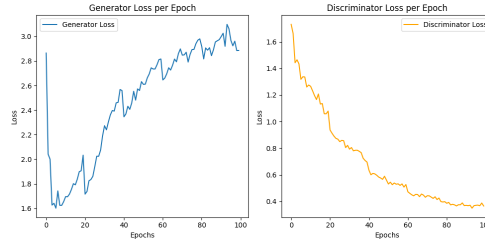


Figure 18: Loss, 40 species

The result that can be seen on Figure 20 and Figure 21, is from another training run on 40 bird species, where a different input image transformation was used (many images appear cropped and drastically altered) and the generator's learning rate was significantly increased (above 0.0007).

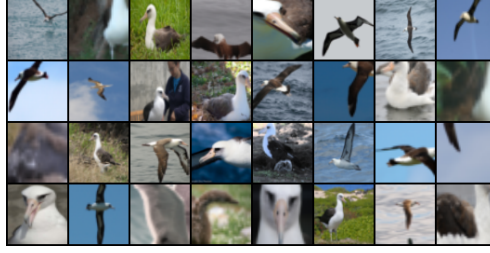


Figure 19: Real images



Figure 20: Stage 1

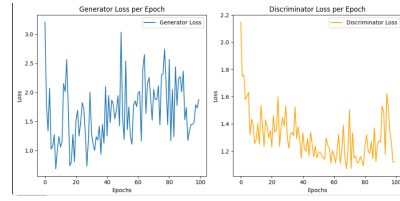


Figure 21: Greške u prvoj fazi

Regarding the training of the Stage 2 for 40 bird species, unfortunately, due to technical limitations, I was unable to complete it fully.

5 Testing Phase

Regarding the testing phase, it has been implemented and operates on an independent test dataset. Unfortunately, the results obtained from the generated model are unsatisfactory. The model has not genuinely learned to recognize the relevant characteristics of birds. The results of testing on the dataset of 40 bird species, using the obtained Stage 1 generator model can be seen on Figure 22.

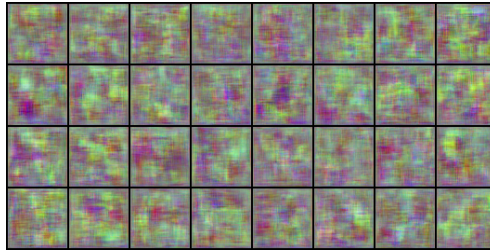


Figure 22: Test, Stage 1

6 Results and Conclusion

During training, I was unable to achieve satisfactory results, i.e., to generate images of birds based on the given textual descriptions. The images obtained at the end of the first-stage training resemble the original bird images in terms of colors and discernible shapes. Additionally, due to technical issues and the model's complex architecture, I failed to obtain results for the second stage when using a larger number of species or images. Based on the second-stage results for a single species, I concluded that

the Stage 2 successfully recognizes basic elements in the Stage 1 images and further enhances them. In certain cases, the model manages to learn some characteristics, while in others it does not. For example, during training, for a description "a small bird with a gray head, brown wings, and a gray and brown belly", the resulting generated image can be observed on Figure 23.



Figure 23: small bird with gray head

During training, I encountered numerous issues, such as the imbalance between generator and discriminator training, insufficient data volume, and Mode Collapse. Initially, I began training with all 200 bird species, but I soon realized that the training process would be too time-consuming and consequently reduced the total number of species to only 10. After some time, I concluded that the model might not be successfully learning the characteristics of the birds due to insufficient data. I therefore gradually increased the number of species, first to 20 and then to 40. Along the way, I also attempted to fine-tune the model for just a single species. I concluded that the model correctly learns colors and image features for one species, and that a higher learning rate significantly contributes to better training in this case. I also observed how crucial proper weight initialization is. Additionally, I experimented with and monitored the model's behavior for different loss calculations.

I also faced the issue known as Mode Collapse. This occurs when the generator begins to produce only one or a few similar images because this specific generator output minimizes the loss relative to the discriminator. As a result, instead of generating a wide variety of different images, the generator continually produces similar or identical images, so that by the end of the training all generated images become the same.

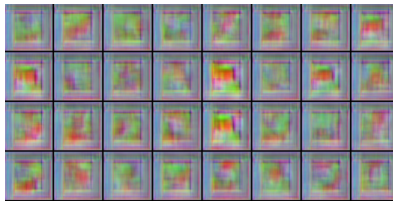


Figure 24: Mode collapse

In addition to all of the above, I observed that problems often arise when one part of the model, typically the discriminator, becomes too accurate while the other fails to achieve a similar level of precision. This issue was the most challenging, and I attempted to resolve it in several ways, by modifying key parameters such as the learning rate (for both the generator and discriminator), `GF_DIM`, `DF_DIM`, and `Z_DIM`, as well as by trying different optimizers.

By tuning these parameters, I managed to achieve a somewhat better balance between the generator and discriminator, which improved the overall stability and performance of the model. Although I did not achieve the expected results, I reached

numerous conclusions while working on GAN networks. The main conclusions I derived are as follows:

- A higher generator learning rate produces images with clearer and more defined shapes and colors, whereas a lower rate yields more abstract, art-like images.
- A lower discriminator learning rate allows the discriminator to learn more slowly, helping to prevent a situation where the discriminator overpowers the generator.
- A higher value of the GF_DIM parameter, which represents the base number of filters in the first layer of the generator, produces images with more detail and texture. The higher its value, the greater the model's capacity to learn complex features. An excessively high value of this parameter can lead to increased model complexity and training difficulties.
- A higher value of the DF_DIM parameter, which represents the base number of filters in the first layer of the discriminator, helps the discriminator to more precisely distinguish real from fake images.
- A higher value of the Z_DIM parameter, which represents the dimensionality of the latent vector (noise), allows the generator to use a broader and more complex latent space. This increases the diversity and detail of generated images, enabling it to capture more variations in characteristics. However, it simultaneously increases computational demands and training complexity due to processing a larger number of dimensions.
- The initialization of network weights significantly influences the generated images. Proper initialization can accelerate convergence, avoid local minima, and stabilize the training process, while poor initialization can result in low image quality and lack of diversity.

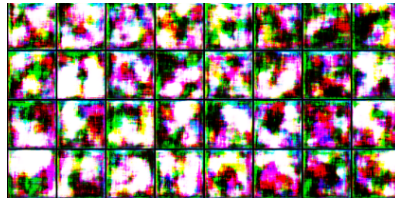


Figure 25: Poor Weights Initialization

- Different input image transformations contribute to greater diversity and improve model learning. However, overly drastic transformations can significantly alter the input images (e.g., large color differences, cropping, and similar changes) to such an extent that neither a human nor the model can recognize that the image contains a bird.
- Using the SGD optimizer for the discriminator instead of Adam can lead to unstable training, where all generated images begin to resemble each other, resulting in the mode collapse phenomenon. The idea behind using SGD was to slow down the discriminator's convergence, giving the generator more time to adapt and achieve a better balance between the two models. In practice, however, this choice of optimizer contributed to even greater instability between the generator and discriminator.

Throughout this work, I concluded that working with GAN models is an extremely complex topic that requires thorough understanding and careful parameter tuning. Training GANs is quite challenging when it comes to establishing balanced training between the generator and discriminator, and it is definitely a topic on which further work and improvements can always be done.

References

- [1] <https://github.com/zeusm9/Text-to-Photo-realistic-Image-Synthesis-with-Stacked-Generative-Adversarial-Networks/tree/master>
- [2] <https://github.com/savya08/StackGAN/tree/master>
- [3] Cristian Bodnar, *Text to Image Synthesis Using Generative Adversarial Networks*. University of Manchester, 2018.
- [4] Han Zhang, Tao Xu, Hongsheng Li³, Shaoting Zhang, Xiaogang Wang, Xiao lei Huang, Dimitris Metaxas *StackGAN: Text to Photo-realistic Image Synthesis with Stacked Generative Adversarial Networks*. Rutgers University, Lehigh University, The Chinese University of Hong Kong, Baidu Research